

P3=B4 XOR B5 XOR B6 XOR B7
---eq2

This is referred to as (7,4) Hamming code.

Implementation overview (with code walkthrough)

I have written a Python code to implement the algorithm. Here, *even parity* is used.

Data is the input data stream taken as input from the user (example: data='1101').

'm' is the length of data taken as input from user (m=4).

Encoding of data

Step 1: *calculateRedundantBitSize()* function finds the value of 'r'; r=3.

Step 2: In *createEncodedDat()*, the *encodedList* is the list initialised with all 0s.

For (7,4), the Hamming code *encodedList* will look like this:

0	0	0	0	0	0	0
---	---	---	---	---	---	---

The next code copies data bits to specific locations:

```
for i in range(1,m+r+1,1):
    if(i==(2**j)):
        j=j+1
    else:
        encodedList[i-1]=data[k]
        k=k+1
```

The *for* loop is iterating over the complete data length of 'm+r'. If 'i' is power of 2 in the *encodedList*, the i-1 index will hold the data bit. For (7,4), the Hamming code *encodedList* will look like this:

0	0	1	0	1	0	1
---	---	---	---	---	---	---

Then the parity bits are calculated as follows:

```
for j in range(0,r,1):
    parityBitLoc = 2**j
    for i in range(1,m+r+1,1):
        if((parityBitLoc & i) != 0):
            p=parityBitLoc-1
            encodedList[p]= str(int(encodedList[i-1]) ^
int(encodedList[p]))
```

'j' is iterating over parity bits and 'i' is iterating over the full data stream. The condition:

```
if((parityBitLoc & i) != 0)
```

...will verify the specific positions for parity bit calculation, as specified in equation 2. 'p' is indicating the position of the parity bit in the list.

For (7,4) Hamming code, the *encodedList* will look like this:

1	0	1	0	1	0	1
---	---	---	---	---	---	---

Finally, the encoded bits are stored as a string in *encodedData*.

Decoding and error correction

We now look at error correction. For error simulation, users can give an input to intentionally alter a bit. *rcvdList* is an erroneous *bitString*, which is passed on to the *decodedData* function.

As an example, we assume that bit 4 is corrupted. *rcvdList* will look like this:

1	0	1	1	1	0	1
---	---	---	---	---	---	---

In the *decodedData* function, execute the following code:

```
chkList[p] = int(rcvdList[i-1]) ^ int(rcvdList[p]) ^
int(chkList[p])
```

After execution of this code, *chkList* will look like this:

P1	P2		P3			
0	0	0	1	0	0	0

We convert it to a binary:

$P3P2P1 = (100)_2$

...which is 4 in decimals. This matches exactly with the user input. Thus it can also detect and correct the error. You can find the full source code on GitHub at <https://github.com/SupriyoGanguly/python-hamming>.

Hamming code is a cost-effective solution for detecting and correcting single-bit errors in computers. But if multiple bits are erroneous, it can totally spoil the data. Satellites and modem communications use *hamming* code. The simple Python code implementation discussed in this article can be used as a module by programmers for *hamming* code encoding and decoding for any standard data stream length of 4 bits, 7 bits, 13 bits, and so on. **END** 🐧

By: Supriyo Ganguly

The author is an M. Tech in embedded systems, and works as a senior technical officer at the Electronics Corporation of India Limited, Hyderabad.